

# 尚硅谷嵌入式项目之 boomPI

## 第 3 章 操作系统简介

独立审阅稿 v0.1

2026-08-10

## 第 3 章 操作系统简介

上一章已经把供电、存储、网络、摄像头和屏幕连接到 RV1106G3。电路接通以后，应用程序仍然不能把某个引脚直接当成网口、文件或画面来使用。它还需要有人安排多个工作谁先运行、数据放在哪里、硬件有新消息时怎样通知程序。承担这些公共工作的软件就是操作系统。

在这门课程里，“Linux 内核”专指负责管理处理器、内存和设备的核心部分；“Linux 系统”还包括应用运行所需的库、配置和文件。引导程序位于 Linux 之前，负责把内核带到可以运行的位置。本章先把这三层关系理顺，下一章再登录 boomPI 查看真实系统。

### 3.1 硬件已经接好，为什么还需要操作系统

原理图能确认处理器与 eMMC、网口、摄像头和屏幕之间有电气连接，却不能决定多个程序怎样轮流使用处理器，也不能防止两个程序同时改写同一块硬件。连接是硬件条件，分配和保护是软件问题。

操作系统位于应用与硬件之间。Linux 内核安排处理器时间和内存，通过驱动管理具体设备；应用只提出“读取数据”“保存文件”或“申请内存”这样的请求，不需要自己寻找每一个引脚和寄存器。图 3-1 只看一条路线：应用先到内核，内核再通过驱动到达硬件，处理结果再沿原路返回。

#### 应用怎样到达硬件

应用提出请求，Linux 内核安排资源，设备驱动完成具体操作

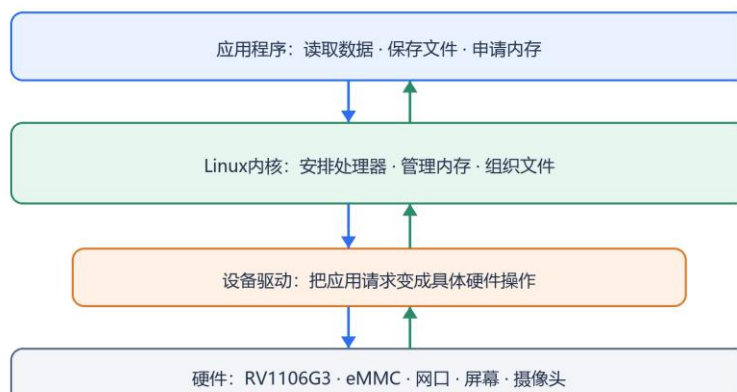


图 3-1 应用经 Linux 内核和驱动使用硬件。教学关系图，不代表全部资源已通过实测。

boomPI 同时连接存储、网络、屏幕、触摸和摄像头。这些资源的使用方法不同，却都需要处理器时间、运行内存和错误处理。Linux 把共同工作集中起来，应用才不必为每一种硬件重新写一套管理方法。

## 3.2 操作系统怎样把硬件交给程序

先看处理器。程序运行时，处理器沿着一条条指令向前执行；如果系统里还有别的工作，内核就要记录哪些工作可以继续，并安排它们依次获得处理器。这个安排过程叫调度。具体怎样选择下一项工作由内核完成，应用不必自己抢占处理器。

再看内存。程序开始运行以后，代码、变量和正在处理的数据都要放进运行内存。内核给不同程序安排可用区域，也在程序结束后回收。应用只提出需要多少空间，不需要知道每一颗存储单元的物理位置。

最后看外设。网口、屏幕和摄像头的控制方式各不相同，内核里负责具体硬件的部分叫设备驱动。应用通过约定好的入口提出请求，驱动再把请求变成控制器能够执行的操作。Linux 5.10 官方把这些面向应用的入口整理在[用户空间 API](#)中，把设备与驱动的关系整理在[设备模型文档](#)中。这里引用的是上游 Linux 5.10 概念文档，不代表 boomPI 厂商 5.10.160 内核与上游实现完全相同；某个设备是否已经绑定驱动、能否实际工作，仍要在板子启动后逐项确认。

## 3.3 多个程序为什么能在同一段时间内向前运行

存放在 eMMC 中的应用只是一个程序文件。Linux 把它装入内存并开始执行以后，这一次运行才叫进程。同一个程序可以先后启动多次，每次运行都有自己的进程身份和数据。

一个进程可以只沿一条执行路线工作，也可以在内部安排多条路线，这些路线叫线程。同一进程内的线程共享代码和大部分数据。它们怎样协调同一份数据属于后面的项目开发内容，本节只分清三层关系：eMMC 里保存的是程序文件，开始运行后形成进程，进程内部可以有一条或多条线程。

操作系统在可以运行的进程和线程之间分配处理器时间；某项工作等待数据时，先让出处理器，事件到达后再继续。多个工作在同一段时间内交替推进，称为并发，不表示它们在同一个瞬间都占用同一颗处理器。图 3-2 中的 A、B、C 只是帮助理解轮流运行，不代表 boomPI 当前项目已经采用三进程架构。

## 一颗处理器怎样让多个任务向前推进

操作系统轮流分配处理器时间；等待数据的任务先让出处理器



图 3-2 任务 A、B、C 轮流使用处理器时间。教学示意，不代表项目的实际进程划分。

Linux 5.10 的[调度器设计文档](#)会继续讲运行时间怎样记录、下一项任务怎样选择。初学阶段不需要记算法，只要先明白：看到多个程序都在工作，不等于它们必须在同一瞬间一起占用处理器；看到某次 CPU 占用较高，也不能据此断言程序已经卡死或满足实时要求。

## 3.4 内存、中断和文件系统各管哪一段

程序从 eMMC 读出后，需要放进运行内存才能执行；计算中的图像、网络数据和界面状态也暂存在这里。断电后这些内容通常会消失，所以运行内存不负责长期保存。内核给不同进程安排可用区域，并在进程结束后回收。

外设有新事件时，不必让处理器一直反复询问。网口收到数据、定时器到点或某个设备完成工作，都可以发出信号提醒处理器，这种提醒叫中断。内核与驱动先接住硬件事件，再让等待结果的程序继续运行。Linux 5.10 的[通用中断文档](#)还会继续区分不同的处理方式，本章不展开这些内核开发细节。

eMMC 能够按位置读写数据，却不知道哪些内容属于程序、配置或图片。文件系统给这些数据建立名称、目录和访问规则。Linux 把目录组织成一棵从根目录开始的树；Linux 5.10 的[虚拟文件系统文档](#)说明不同文件系统怎样向应用提供共同入口，具体目录和查看命令留到下一章。

内存保存当前运行需要的数据，中断把外设事件送到内核，文件系统组织需要长期保留的内容。板子刚上电时，Linux 内核的内存管理、中断处理和文件系统尚未开始工作，内核必须先被装入内存并找到根文件系统。

### 3.5 从上电到应用出现，系统经历了什么

电源稳定并解除复位后，芯片内部固化的启动代码先寻找后续启动内容，这段代码叫 BootROM。随后，引导程序把 Linux 内核装入内存，并把执行权交给内核。RV1106 资料列出了 eMMC、SD/MMC 等启动能力，却没有给出 boomPI 当前配置的完整优先顺序，因此这里只讲各阶段职责，不虚构失败回退路线。

内核开始运行后，会建立内存管理和任务调度，启动必要的驱动，并找到根文件系统。根文件系统不是一块新的芯片，而是 Linux 启动后需要的一套目录、命令、库和配置文件。内核随后启动第一个用户空间程序，通常称为 init；init 再按照系统配置启动服务和应用。Linux 5.10 关于[“找不到可用 init”](#)的说明，也从故障角度列出了根文件系统、程序文件、解释器和依赖库之间的关系。

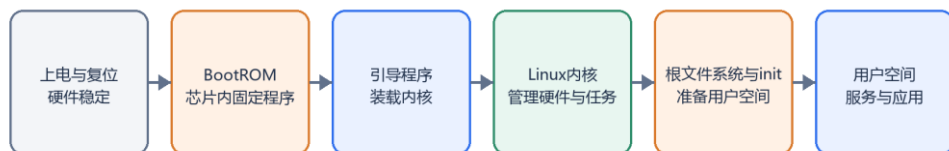


图 3-3 从硬件稳定到用户空间应用的交接过程。教学示意。

图 3-3 中每一格只完成自己的职责。BootROM 不是完整操作系统，引导程序不负责长期运行应用；Linux 内核也不会自动包含所有命令和配置。出现启动故障时，先判断停止在哪个阶段，比把所有黑屏或无登录提示都归为“系统坏了”更有效。

本轮板端身份快照观察到 Linux 5.10.160、armv7l 和 Buildroot 2023.02.6，进程快照中 PID 1 显示为 init。这些结果只说明当时运行系统的身份与一次进程状态，不证明 BSP 已经能够从源码完整复现，也不说明所有外设已经可用。

系统完成启动后，处理器时间、运行内存和已绑定驱动的设备由内核统一管理；其他外设是否可用仍需逐项测试。下一章先登录 boomPI，在真实终端观察目录、进程和存储空间；烧录和硬件测试放在后续操作章节。